

Cursor Tutorial: Build a Web-Based 五子棋 Game

Lesson Title

Building Your First Web Game MVP with Cursor: 五子棋 / Gomoku

Student Level

Middle school and high school beginners

Final Goal

Students will use Cursor to create a simple clickable web game:

五子棋 / Gomoku / Five-in-a-Row

The first version does not need to be perfect. The goal is to build a working MVP that students can test and improve.

1. Lesson Connection to the Course Material

In the attached Lesson 04 slides, students learn that today's goal is to create a **first clickable MVP**, not a perfect final product. The slides also teach the construction workflow:

Prompt → Generate → Test → Improve

This Gomoku project follows the same idea.

Instead of asking Cursor to build a huge perfect game immediately, students will build it in small steps:

1. Create a project folder.
2. Generate a starter web app.
3. Build one working page.
4. Include one main interaction.
5. Test the game.
6. Make a bug list.
7. Improve the game step by step.

This matches the "Construction Checklist" and "First MVP" ideas from the course slides.

2. What Students Are Building

What Is 五子棋?

五子棋 is also called **Gomoku** or **Five-in-a-Row**.

It is a two-player board game.

Simple Rules

1. The game uses a grid board.
2. Two players take turns.
3. Black moves first.
4. White moves second.
5. A player wins by placing five stones in a row.
6. The five stones can be:
 - horizontal
 - vertical
 - diagonal

3. MVP Version for This Lesson

The course material says an MVP means:

- **Minimum:** only the most important features
- **Viable:** it actually works
- **Product:** something you can show and test today

For this lesson, the Gomoku MVP will include:

Feature	Included in MVP?
15 × 15 board	Yes
Two players	Yes
Black and white stones	Yes
Click to place stone	Yes
Show whose turn it is	Yes
Detect five in a row	Yes
Restart button	Yes
Computer AI opponent	No
Online multiplayer	No
User login	No

Feature	Included in MVP?
Score history	No

Students should understand that **not included** does not mean “bad.” It means we are keeping the first version simple enough to finish.

4. Project Folder

Ask students to create a folder called:

```
gomoku-game
```

Inside the folder, Cursor will help create:

```
gomoku-game/  
  index.html  
  style.css  
  script.js  
  README.md
```

Use the house-building idea from the course slide:

File	Meaning	Job
<code>index.html</code>	Structure	Creates the page and game board container
<code>style.css</code>	Design	Makes the board, stones, button, and layout look nice
<code>script.js</code>	Behavior	Handles clicks, turns, win checking, and restart
<code>README.md</code>	Instructions	Explains what the project is and how to run it

5. Important Mindset: AI Helps, You Lead

Cursor can generate code quickly and help fix errors. Cursor describes its product as using agents that can turn ideas into code and understand your codebase. ([Cursor](#))

But students must stay in control.

The course slide says:

```
AI helps, but YOU lead.
```

That means students should:

- Decide what the app should do.
- Test if the app works.
- Ask questions when they do not understand.
- Never accept code they do not generally understand.
- Improve one thing at a time.

Cursor's own agent best-practice guidance also recommends starting with a plan, giving clear goals, and using testing or review signals to check whether changes are correct. ([Cursor](#))

6. The Starter Prompt Formula

The course material gives a useful starter prompt formula:

```
Act as a web developer.  
Build a [App Name].  
It is for [User].  
It needs to [Main Action].  
Make it look [Colors/Vibe].  
Use simple HTML/CSS/JS.  
Keep it beginner-friendly.
```

For this project, students will turn that formula into a Gomoku prompt.

7. Step-by-Step Cursor Workflow

Step 1: Ask Cursor for a Plan First

Do not ask Cursor to code immediately.

First, ask Cursor to make a plan.

Prompt 1: Project Plan

```
Act as a web development teacher for middle and high school students.  
  
I want to build a simple web-based game called 五子棋, also known as Gomoku or  
Five-in-a-Row.  
  
Before writing any code, please create a beginner-friendly project plan.  
  
The game should be a first MVP, not a perfect final product.  
  
Requirements:
```

- Use only HTML, CSS, and JavaScript.
- The game should run in a web browser.
- Use a 15x15 board.
- Two players play on the same computer.
- Black moves first.
- White moves second.
- A player clicks an empty cell to place a stone.
- A player wins when they get 5 stones in a row.
- The game should check horizontal, vertical, and diagonal wins.
- Add a Restart Game button.

Please explain:

1. What files we need
2. What each file does
3. What the main user interaction is
4. What the minimum MVP features are
5. What features we should NOT build yet
6. The step-by-step implementation plan

What Students Should Learn

After Cursor answers, students should identify:

- Main user: two students playing on one computer
- Main action: click a cell to place a stone
- Main logic: check five stones in a row
- MVP boundary: no AI opponent yet

Step 2: Ask Cursor to Create the Starter Files

Now students can ask Cursor to generate the app.

Prompt 2: Generate the First MVP

Now build the first working MVP of the web-based Gomoku game.

Please create these files:

- index.html
- style.css
- script.js
- README.md

Use plain HTML, CSS, and JavaScript only.

Game requirements:

- Create a 15x15 board.
- Each cell should be clickable.
- Black player moves first.
- White player moves second.
- Show the current player's turn.

```
- Do not allow a player to place a stone on an occupied cell.
- After each move, check if the current player has won.
- A player wins with 5 stones in a row horizontally, vertically, diagonally down-right, or diagonally up-right.
- Show a clear winner message.
- Stop the game after someone wins.
- Add a Restart Game button.

Design requirements:
- Center the game on the page.
- Make the board look clean and easy to see.
- Use black and white circular stones.
- Make the layout beginner-friendly.

Code requirements:
- Add helpful comments.
- Keep the code simple.
- Avoid advanced frameworks.
- Explain briefly how to run the game in the README.
```

Step 3: Run and Preview the App

Students should now test the game.

How to Run

1. Open the `gomoku-game` folder in Cursor.
2. Find `index.html`.
3. Right-click it and open it in a browser, or use a simple local server if available.
4. Play the game.

Student Testing Checklist

Test	Expected Result
The page opens	The game title and board appear
Board has 15 rows and 15 columns	The grid looks correct
Click empty cell	A stone appears
First move	Black stone appears
Second move	White stone appears
Click same cell twice	Second stone is not added
Five black stones in a row	Black wins
Five white stones in a row	White wins
Click after win	No new moves

Test	Expected Result
Restart button	Board clears and black starts again

This follows the course slide's "Inspection: Testing & Logging Bugs" idea: test buttons, check readability, and write down what is broken.

Step 4: Ask Cursor to Explain the Code

Students should not stop after generating code. They should ask Cursor to explain it.

Prompt 3: Explain the Code

```
Please explain this Gomoku project like I am a beginner student.
```

```
Explain in simple English:
```

1. What index.html does
2. What style.css does
3. What script.js does
4. How the 15x15 board is created
5. How the game stores black and white stones
6. How clicking a cell works
7. How the game switches turns
8. How the win-checking function works
9. How the restart button works

```
Please use short examples and avoid advanced vocabulary.
```

Teacher Tip

Ask students:

```
Which file controls the design?  
Which file controls what happens when you click?  
Which file contains the page structure?
```

Expected answers:

- Design: `style.css`
- Click behavior: `script.js`
- Page structure: `index.html`

This directly reinforces the "Name That File" quiz from the lesson slides.

8. Step 5: Improve One Thing at a Time

The course material says that if AI gets confused, students should:

```
Make the app simpler.  
Fix this error step by step.  
Only change one thing at a time.  
Explain this file structure for a beginner.
```

So do not ask Cursor to add many features at once.

Improvement 1: Better Visual Design

Prompt 4: Improve the Design

```
Please improve only the visual design of the Gomoku game.
```

```
Do not change the game rules.
```

```
Design improvements:
```

- Add the title: 五子棋 Gomoku Game
- Make the board look like a wooden board.
- Make black and white stones look smooth and round.
- Make the current player message larger and easier to read.
- Make the Restart Game button look nicer.
- Keep the layout centered.
- Keep the code beginner-friendly.

```
After making changes, explain which file you changed and why.
```

Improvement 2: Add Move Counter

Prompt 5: Add a Move Counter

```
Please add a move counter to the Gomoku game.
```

```
Requirements:
```

- Show "Moves: 0" at the start.
 - Increase the number by 1 after each valid move.
 - Do not increase the number if a player clicks an occupied cell.
 - Reset the move counter to 0 when the Restart Game button is clicked.
 - Keep the code simple and explain what changed.
-

Improvement 3: Add Draw Detection

A 15 × 15 board has 225 cells. If all cells are filled and nobody wins, the game is a draw.

Prompt 6: Add Draw Detection

Please add draw detection to the Gomoku game.

Requirements:

- If all 225 cells are filled and no player has won, show "Draw game!"
- Stop the game after a draw.
- The Restart Game button should still work.
- Keep the existing winner detection working.
- Explain the logic in simple English.

Improvement 4: Highlight the Winning Stones

This is a good challenge for stronger students.

Prompt 7: Highlight the Winning Line

Please improve the winner detection so that the five winning stones are highlighted.

Requirements:

- When a player wins, highlight the 5 stones that created the win.
- Use a simple visual effect such as a border or glow.
- Do not change the basic game rules.
- Keep the code beginner-friendly.
- Explain how the code finds and highlights the winning stones.

9. Debugging Prompts

Students must learn how to ask for help clearly.

Bad Prompt

It does not work. Fix it.

Good Prompt Template

My Gomoku game has a bug.

Problem:

[Describe exactly what happens.]

Expected behavior:

[Describe what should happen.]

Actual behavior:

[Describe what actually happens.]

Error message:

[Paste browser console error here, if any.]

Please help me fix this step by step.

Only change the files that are necessary.

Explain the cause in simple English.

Example Debug Prompt 1: Board Does Not Appear

My Gomoku game has a bug.

Problem:

The page opens, but the 15x15 board does not appear.

Expected behavior:

I should see a 15x15 game board.

Actual behavior:

I only see the title and restart button.

Please check index.html, style.css, and script.js.

Help me fix this step by step.

Explain the cause in simple English.

Example Debug Prompt 2: Stones Do Not Appear

My Gomoku game has a bug.

Problem:

When I click a cell, no stone appears.

Expected behavior:

A black or white stone should appear in the clicked cell.

Actual behavior:

The board is visible, but clicking does nothing.

Please check the click event logic in script.js.
Explain the problem and fix it.

Example Debug Prompt 3: Winner Is Not Detected

My Gomoku game has a bug.

Problem:
I placed five black stones in a horizontal row, but the game did not say Black wins.

Expected behavior:
The game should show a winner message when five stones are in a row.

Actual behavior:
The game continues with the next turn.

Please review the winner detection logic.
Check horizontal, vertical, and diagonal directions.
Fix the bug and explain the solution.

10. Student Bug Log

Ask students to keep a simple bug log, as suggested in the lesson slides.

Bug Number	What Happened?	Which File Might Be Wrong?	Fixed?
1	Board did not show	script.js or style.css	Yes / No
2	Button looked ugly	style.css	Yes / No
3	Restart did not work	script.js	Yes / No
4	Title was missing	index.html	Yes / No

11. Classroom Activity: Think-Pair-Share

Use this activity from the lesson material and adapt it to Gomoku.

Think

Ask each student:

What is the one main action the user must do in the Gomoku game?

Expected answer:

Click a cell to place a stone.

Pair

Students explain their app in one sentence to a partner.

Example:

My app is a two-player Gomoku game where players click cells to place stones and try to get five in a row.

Share

Students answer:

Is this MVP simple enough to build today?
What feature should we save for later?

Features to save for later:

- AI opponent
- online multiplayer
- login
- leaderboard
- animations
- sound effects

12. Complete Student Prompt Set

Here is the full sequence students can use in Cursor.

Prompt A: Plan

Act as a web development teacher for middle and high school students.

I want to build a simple web-based game called 五子棋, also known as Gomoku or

Five-in-a-Row.

Before writing any code, please create a beginner-friendly project plan.

The game should be a first MVP, not a perfect final product.

Requirements:

- Use only HTML, CSS, and JavaScript.
- The game should run in a web browser.
- Use a 15x15 board.
- Two players play on the same computer.
- Black moves first.
- White moves second.
- A player clicks an empty cell to place a stone.
- A player wins when they get 5 stones in a row.
- The game should check horizontal, vertical, and diagonal wins.
- Add a Restart Game button.

Please explain:

1. What files we need
2. What each file does
3. What the main user interaction is
4. What the minimum MVP features are
5. What features we should NOT build yet
6. The step-by-step implementation plan

Prompt B: Build

Now build the first working MVP of the web-based Gomoku game.

Please create these files:

- index.html
- style.css
- script.js
- README.md

Use plain HTML, CSS, and JavaScript only.

Game requirements:

- Create a 15x15 board.
- Each cell should be clickable.
- Black player moves first.
- White player moves second.
- Show the current player's turn.
- Do not allow a player to place a stone on an occupied cell.
- After each move, check if the current player has won.
- A player wins with 5 stones in a row horizontally, vertically, diagonally down-right, or diagonally up-right.
- Show a clear winner message.
- Stop the game after someone wins.
- Add a Restart Game button.

Design requirements:

- Center the game on the page.
- Make the board look clean and easy to see.
- Use black and white circular stones.
- Make the layout beginner-friendly.

Code requirements:

- Add helpful comments.
- Keep the code simple.
- Avoid advanced frameworks.
- Explain briefly how to run the game in the README.

Prompt C: Explain

Please explain this Gomoku project like I am a beginner student.

Explain in simple English:

1. What index.html does
2. What style.css does
3. What script.js does
4. How the 15x15 board is created
5. How the game stores black and white stones
6. How clicking a cell works
7. How the game switches turns
8. How the win-checking function works
9. How the restart button works

Please use short examples and avoid advanced vocabulary.

Prompt D: Improve

Please improve only the visual design of the Gomoku game.

Do not change the game rules.

Design improvements:

- Add the title: 五子棋 Gomoku Game
- Make the board look like a wooden board.
- Make black and white stones look smooth and round.
- Make the current player message larger and easier to read.
- Make the Restart Game button look nicer.
- Keep the layout centered.
- Keep the code beginner-friendly.

After making changes, explain which file you changed and why.

Prompt E: Debug

My Gomoku game has a bug.

Problem:

[Describe exactly what happens.]

Expected behavior:

[Describe what should happen.]

Actual behavior:

[Describe what actually happens.]

Error message:

[Paste browser console error here, if any.]

Please help me fix this step by step.

Only change the files that are necessary.

Explain the cause in simple English.

13. Teacher Demonstration Script

You can say this to students:

"Today we are not trying to build the perfect game. We are building the first clickable MVP. Our goal is simple: open the page, click the board, place stones, detect a winner, and restart the game. Cursor is our co-pilot, but we are the pilots. We will not just say 'build me a game' and walk away. We will give clear instructions, test the result, write down bugs, and improve one thing at a time."

14. Student Exit Ticket

At the end of class, ask students to answer:

1. What did Cursor help you build today?
2. What is one thing that worked well?
3. What is one bug or problem you found?
4. Which file controls the design?
5. Which file controls the clicking behavior?
6. What is one feature you want to add next time?

Expected key answers:

- Design: `style.css`
- Clicking behavior: `script.js`
- Structure: `index.html`

15. Optional Homework

Ask students to choose **one** improvement only:

1. Add a move counter.
2. Add draw detection.
3. Highlight the winning five stones.
4. Add a short instruction section.
5. Improve the board design.
6. Add sound when a stone is placed.

Homework rule:

Only improve one thing at a time.
Test after every change.
Write down one bug and how you fixed it.

16. Final Message for Students

A good AI developer is not someone who only writes short prompts.

A good AI developer:

- knows what they want to build,
- starts with a simple MVP,
- gives clear instructions,
- tests carefully,
- fixes bugs step by step,
- and understands the code well enough to explain it.

For this project, Cursor helps build the Gomoku game, but **you are the architect**.